

Apprendimento Automatico

Riassunto Ridotto — Parte Meo (concetti chiave e formule)

Basato sulle slide del corso (da Zhou Zhi-Hua, *Machine Learning*)

A.A. 2025–2026

Indice

Capitolo 4 — Decision Trees (Alberi di Decisione)	3
Processo di base	3
Misure di purezza (impurity measures)	3
Information Gain e algoritmo ID3	3
Gain Ratio e algoritmo C4.5	4
Indice di Gini e algoritmo CART	4
Pruning (potatura)	4
Feature continue e valori mancanti	5
Alberi di decisione multivariati	5
Capitolo 5 — Neural Networks (Reti Neurali)	6
Cenni storici	6
5.1 Il modello di neurone (M-P Neuron Model)	6
Funzioni di attivazione	6
5.2 Percettrone e reti multi-strato	7
5.3 Algoritmo di Error Backpropagation (BP)	7
BP standard vs BP accumulato	8
Capacità rappresentativa e overfitting	8
Capitolo 7 — Bayes Classifiers (Classificatori Bayesiani)	9
Bayes Decision Theory (teoria delle decisioni bayesiana)	9
Stima di Massima Verosimiglianza (MLE)	10
Naïve Bayes Classifier	10
Semi-Naïve Bayes (cenni)	11
Reti Bayesiane	11
EM Algorithm (Expectation-Maximization)	12
Capitolo 9 — Clustering	13
Task del Clustering	13
Misure di Validità (Performance Measure)	13
Misure di Distanza	14
K-Means Clustering	14
Learning Vector Quantization (LVQ)	15
Mixture-of-Gaussian Clustering	15
Clustering Gerarchico — AGNES	16
Clustering basato sulla densità — DBSCAN	16
Capitolo 10 — Dimension Reduction and Metric Learning (Riduzione della Dimensionalità)	17
k-Nearest Neighbor Learning (kNN)	17
Curse of Dimensionality e Low-dimensional Embedding	17
Multiple Dimensional Scaling (MDS)	17

Principal Component Analysis (PCA)	18
Kernelized PCA (KPCA)	19
Manifold Learning	19
Metric Learning	19
Capitolo 11 — Feature Selection and Sparse Learning (Selezione delle Feature)	21
Introduzione	21
Subset Search & Subset Evaluation	21
Filter Methods — Relief	21
Wrapper Methods — LVW	22
Embedded Methods e Regularizzazione L1/L2	22
Sparse Representation e Dictionary Learning	23
Cenni: Compressed Sensing e Matrix Completion	23
Conclusione	24
Capitolo 12 — Computational Learning Theory (Teoria dell'Apprendimento Computazionale)	25
Concetti Base	25
PAC Learning	25
Spazio di Ipotesi Finito ($ \mathcal{H} < \infty$)	26
VC Dimension	26
Rademacher Complexity	27
Stabilità	28
Capitolo 13 — Semi-Supervised Learning (Apprendimento Semi-Supervisionato)	29
Concetti base e assunzioni	29
Metodi Generativi (Generative Methods / EM)	29
Semi-Supervised SVM (S3VM)	30
Graph-based SSL	30
Disagreement-based Methods (Co-training)	31
Semi-Supervised Clustering	31
Capitolo 15 — Rule Learning (Apprendimento di Regole)	32
Concetti Base	32
Sequential Covering	32
Pruning Optimization	33
First-Order Rule Learning — FOIL	33
Cenni a Inductive Logic Programming (ILP)	34

Capitolo 4 — Decision Trees (Alberi di Decisione)

Processo di base

Un albero di decisione classifica un esempio con una sequenza di test, ciascuno su una feature, che partono dalla radice e portano a una foglia (la classe predetta). La costruzione è **ricorsiva e greedy** (algoritmo divide-et-impera): a ogni nodo si sceglie la feature migliore su cui splittare (Split Selection) e si creano i figli. La ricorsione si ferma (nodo diventa foglia) in due casi: (1) tutti i campioni nel nodo appartengono già alla stessa classe, oppure (2) l'insieme di feature disponibili è vuoto o tutti i campioni hanno gli stessi valori di feature (in tal caso la foglia prende l'etichetta di **classe maggioritaria**). L'obiettivo è ottenere un albero che generalizzi bene su dati mai visti, non solo che classifichi perfettamente il training set.

Misure di purezza (impurity measures)

A ogni split si vuole aumentare la **purezza** dei nodi figli, cioè far sì che contengano campioni sempre più omogenei rispetto alla classe. Le misure classiche (nel caso binario, con p^+ e p^- probabilità/proporzioni delle due classi) sono: la **minority class** $\min\{p^+, p^-\}$ (errore se si etichetta con la classe maggioritaria), l'**indice di Gini** $2p^+p^-$ e l'**entropia**. La forma generale (multi-classe) dell'entropia, usata nelle slide, è:

$$Ent(D) = - \sum_{k=1}^{|Y|} p_k \log_2 p_k$$

dove p_k è la proporzione (frequenza relativa) della classe k nel dataset D e $|Y|$ è il numero di classi (per convenzione $p \log_2 p = 0$ se $p = 0$). $Ent(D)$ vale 0 quando il nodo è perfettamente puro (una sola classe) ed è massima, pari a $\log_2 |Y|$, quando le classi sono equidistribuite: più bassa è l'entropia, più alta è la purezza.

Information Gain e algoritmo ID3

Data una feature discreta a con V valori possibili $\{a^1, \dots, a^V\}$, lo split su a produce V nodi figli $D^1, \dots, D^V$. Il **guadagno informativo** (information gain) è:

$$Gain(D, a) = Ent(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} Ent(D^v)$$

Il termine $Ent(D)$ è l'entropia del nodo padre prima dello split; la sommatoria è l'entropia media dei figli, pesata con $\frac{|D^v|}{|D|}$ (la frazione di campioni finiti in ciascun figlio, che ne rappresenta l'importanza). $Gain(D, a)$

misura quindi quanta impurità viene rimossa splittando su a : più è alto, più lo split è utile. **ID3** (Quinlan, 1986) sceglie a ogni nodo la feature con il massimo information gain. Limite noto: il criterio è **biased verso feature con molti valori possibili** (es. un ID univoco per riga darebbe gain massimo ma zero potere di generalizzazione).

Gain Ratio e algoritmo C4.5

Per correggere il bias dell'information gain, C4.5 usa il **gain ratio**:

$$\text{Gain_ratio}(D, a) = \frac{\text{Gain}(D, a)}{IV(a)}, \quad IV(a) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

$IV(a)$ (intrinsic value) è l'entropia della sola distribuzione dei valori di a : cresce quando a ha molti valori possibili (partizioni molto frammentate), quindi dividere per $IV(a)$ penalizza le feature con troppi valori. Il gain ratio è però **biased verso feature con pochi valori**, quindi **C4.5** (Quinlan, 1993) usa un'euristica: sceglie la feature con il gain ratio più alto **tra quelle il cui information gain è superiore alla media** delle feature candidate.

Indice di Gini e algoritmo CART

Il **valore di Gini** di un dataset D è:

$$\text{Gini}(D) = \sum_{k=1}^{|Y|} \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_{k=1}^{|Y|} p_k^2$$

Rappresenta la probabilità di etichettare erroneamente un campione se lo si classificasse a caso secondo la distribuzione delle classi in D : più basso è $\text{Gini}(D)$, più D è puro. L'**indice di Gini di una feature a** è la media pesata dei valori di Gini dei figli:

$$\text{Gini_index}(D, a) = \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Gini}(D^v)$$

CART (Breiman et al., 1984) seleziona come feature di split quella che **minimizza** $\text{Gini_index}(D, a)$: $a_* = \arg \min_{a \in A} \text{Gini_index}(D, a)$.

Pruning (potatura)

Il pruning è la strategia principale contro l'**overfitting** (troppi rami inseguono peculiarità del training set che non generalizzano); un albero troppo piccolo, al contrario, rischia l'**underfitting**. Per valutare l'effetto della potatura si usa tipicamente l'**hold-out**: si riserva un validation set separato dal training set. Esistono due strategie:

- **Pre-pruning**: durante la costruzione, prima di confermare uno split si confronta l'accuratezza di validazione con e senza lo split (nodo lasciato foglia con etichetta = classe maggioritaria). Se lo split non migliora (o peggiora) l'accuratezza di validazione, viene bloccato e il nodo resta foglia. È una procedura top-down eseguita man mano che l'albero cresce. **Vantaggi**: riduce il rischio di overfitting e i costi computazionali. **Svantaggi**: per la sua natura greedy rischia l'**underfitting**, perché uno split localmente inutile potrebbe abilitare split successivi molto utili che non vengono mai esplorati.

- **Post-pruning**: si lascia crescere l'albero completo, poi si esaminano i nodi interni **dal basso verso l'alto** (bottom-up), sostituendo un sottoalbero con una foglia (classe maggioritaria) se ciò non peggiora (o migliora) l'accuratezza di validazione. In generale il post-pruning mantiene più rami del pre-pruning ed è **meno soggetto a underfitting**, con migliore capacità di generalizzazione, ma è più costoso perché richiede di costruire prima l'albero completo e poi valutare ogni nodo interno.

Feature continue e valori mancanti

Per una feature **continua** a con n valori distinti osservati e ordinati $a^1 < \dots < a^n$, si usa una **strategia di bipartizione** (discretizzazione binaria): dato un punto di split t , D è diviso in D_t^- (valori $\leq t$) e D_t^+ (valori $> t$). I candidati sono i punti medi tra valori consecutivi:

$$T_a = \left\{ \frac{a^i + a^{i+1}}{2} \mid 1 \leq i \leq n-1 \right\}$$

e si sceglie il $t \in T_a$ che massimizza il gain: $\text{Gain}(D, a) = \max_{t \in T_a} \text{Gain}(D, a, t)$, dove $\text{Gain}(D, a, t)$ è calcolato come il gain standard usando D_t^- , D_t^+ al posto dei D^v .

Per i **valori mancanti** si introduce un peso w_x per ogni campione (inizialmente 1) e una proporzione $\rho = |\tilde{D}|/|D|$ di campioni **senza** valore mancante per la feature a in esame. Il gain diventa:

$$\text{Gain}(D, a) = \rho \times \left(\text{Ent}(\tilde{D}) - \sum_{v=1}^V \tilde{r}_v \text{Ent}(\tilde{D}^v) \right)$$

dove $\text{Ent}(\tilde{D}) = -\sum_k \tilde{p}_k \log_2 \tilde{p}_k$ è calcolata solo sui campioni non mancanti, $\tilde{p}_k$ è la proporzione della classe k tra questi, e $\tilde{r}_v = |\tilde{D}^v|/|\tilde{D}|$ è la proporzione di campioni non mancanti che cadono nel ramo v . In pratica si calcola il gain "come se" i mancanti non esistessero, poi lo si scala per ρ (penalizzando feature con molti mancanti). Per lo split effettivo: se il valore di a per il campione x è noto, x va nel figlio corrispondente senza cambiare peso; se è **mancante**, x viene mandato in **tutti** i figli, con peso ridotto a $\tilde{r}_v \cdot w_x$ in ciascuno (split "probabilistico" pesato).

Alberi di decisione multivariati

Un albero classico ha confini decisionali **assi-paralleli** (ogni test riguarda una sola feature), il che può richiedere molti split per approssimare confini obliqui. Negli **alberi multivariati** ogni nodo interno non è più un test su una singola feature ma un **classificatore lineare** sulla combinazione delle feature: $\sum_{i=1}^d w_i a_i = t$, dove w_i è il peso della feature a_i e sia i pesi w_i che la soglia t sono appresi dai dati disponibili in quel nodo. Questo permette confini decisionali obliqui, più compatti ed espressivi rispetto ai tradizionali split assi-paralleli.

Capitolo 5 — Neural Networks (Reti Neurali)

Cenni storici

Tre fasi principali: (1) 1943 modello **M-P** (McCulloch-Pitts), 1958 **Percettrone** (Rosenblatt); (2) 1982 rete di **Hopfield**, 1986 riscoperta dell'algoritmo **BP** (Rumelhart et al.); (3) dal 2006 **Deep Learning** (Deep Belief Network di Hinton, poi CNN su ImageNet 2012). Utile solo come contesto, raramente oggetto di domande puntuali.

5.1 Il modello di neurone (M-P Neuron Model)

Un neurone riceve n segnali di ingresso $x_1, \dots, x_n$, li combina linearmente pesandoli, confronta la somma con una **soglia** θ e applica una **funzione di attivazione** f :

$$y = f \left(\sum_{i=1}^n w_i x_i - \theta \right)$$

- x_i : segnale in ingresso dall' i -esimo neurone (o attributo di input).
- w_i : **peso di connessione**, indica l'importanza/influenza dell'ingresso x_i .
- θ : **soglia** del neurone; il neurone si "attiva" solo se la somma pesata supera θ .
- f : funzione di attivazione, che trasforma il potenziale netto in output.

Questa è l'unità computazionale base di ogni rete neurale: tutta la rete è una composizione di questi neuroni.

Funzioni di attivazione

- **Funzione a gradino** (ideale, biologicamente motivata): $\text{sgn}(x) = 1$ se $x \geq 0$, altrimenti 0. È discontinua e non derivabile, quindi poco adatta all'apprendimento basato su gradiente.
- **Sigmoide** (usata in pratica): $\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$, continua, derivabile, "schiaccia" i valori reali in $(0, 1)$. La sua derivata ha la comoda forma $f'(x) = f(x)[1 - f(x)]$, sfruttata direttamente nella derivazione del Backpropagation.

5.2 Percettrone e reti multi-strato

Il **perceptrone** è un classificatore binario a due strati (input + un neurone M-P di output). Con pesi e soglia opportuni realizza le funzioni logiche AND, OR, NOT (es. AND: $w_1 = w_2 = 1, \theta = 2$).

Regola di apprendimento del perceptrone: per un campione (x, y) con output prodotto $\hat{y}$, i pesi si aggiornano come

$$w_i \leftarrow w_i + \Delta w_i, \quad \Delta w_i = \eta(y - \hat{y}) x_i$$

- $\eta \in (0, 1)$: **learning rate** (tasso di apprendimento), controlla l'ampiezza dell'aggiornamento.
- Se la predizione è corretta ($y = \hat{y}$) i pesi non cambiano; altrimenti vengono corretti proporzionalmente all'errore ($y - \hat{y}$) e all'input x_i .

Limite fondamentale: un perceptrone a un solo strato converge solo su problemi **linearmente separabili** (AND, OR, NOT) e non può risolvere problemi non separabili come lo **XOR** [Minsky & Papert, 1969]. La soluzione è aggiungere uno o più **strati nascosti (hidden layer)** tra input e output, ottenendo le **reti feed-forward multi-strato**, completamente connesse tra strati adiacenti, in cui l'apprendimento consiste nel tarare pesi e soglie di tutti gli strati.

5.3 Algoritmo di Error Backpropagation (BP)

È l'algoritmo di apprendimento più importante e diffuso per reti multi-strato. Si considera una rete con d neuroni di input, q neuroni nascosti (uno strato) e l neuroni di output, con dataset $D = \{(x_k, y_k)\}_{k=1}^m$.

Notazione: v_{ih} = peso input $\rightarrow$ nascosto; w_{hj} = peso nascosto $\rightarrow$ output; γ_h = soglia neurone nascosto h ; θ_j = soglia neurone di output j .

Passo forward (calcolo dell'output e dell'errore per il campione k):

$$\begin{aligned} \alpha_h &= \sum_{i=1}^d v_{ih} x_i, & b_h &= f(\alpha_h - \gamma_h) \\ \beta_j &= \sum_{h=1}^q w_{hj} b_h, & \hat{y}_j^k &= f(\beta_j - \theta_j) \\ E_k &= \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2 \end{aligned}$$

α_h/β_j sono gli ingressi netti (pre-attivazione) rispettivamente del neurone nascosto h e di output j ; b_h è l'output del neurone nascosto; E_k è l'errore quadratico (somma su tutti i j output) del singolo campione k .

Discesa del gradiente: ogni parametro v si aggiorna come $v \leftarrow v + \Delta v$, con $\Delta w_{hj} = -\eta \partial E_k / \partial w_{hj}$. Applicando la **regola della catena** (poiché E_k dipende da w_{hj} solo attraverso $\hat{y}_j^k$ e β_j):

$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}}, \quad \frac{\partial \beta_j}{\partial w_{hj}} = b_h$$

Sfruttando $f'(x) = f(x)[1 - f(x)]$ (derivata sigmoide) si definisce il **termine di gradiente dello strato di output**:

$$g_j = (y_j^k - \hat{y}_j^k) \cdot \hat{y}_j^k \cdot (1 - \hat{y}_j^k) \quad (5.10)$$

che porta alle regole di aggiornamento finali per lo strato di output:

$$\Delta w_{hj} = \eta g_j b_h \quad (5.11), \quad \Delta \theta_j = -\eta g_j \quad (5.12)$$

Analogamente, propagando l'errore all'indietro fino allo strato nascosto, si definisce il **termine di gradiente nascosto** e_h (da cui il nome "backpropagation": l'errore dello strato di output viene retropropagato pesato dai w_{hj}):

$$e_h = b_h(1 - b_h) \sum_{j=1}^l w_{hj} g_j \quad (5.15)$$

$$\Delta v_{ih} = \eta e_h x_i \quad (5.13), \quad \Delta \gamma_h = -\eta e_h \quad (5.14)$$

- g_j : quanto e in che direzione va corretto l'output j (dipende dall'errore $y_j^k - \hat{y}_j^k$ e dalla pendenza della sigmoide in quel punto).
- e_h : errore "retropropagato" sul neurone nascosto h , ottenuto sommando i g_j di tutti i neuroni di output pesati dalle connessioni w_{hj} che partono da h , modulato dalla derivata della sigmoide $b_h(1 - b_h)$.
- η : se troppo grande causa oscillazioni, se troppo piccolo rallenta la convergenza.

Algoritmo 5.1 (pseudocodice): inizializzare pesi/soglie casualmente in $(0, 1)$; ripetere finché non si soddisfa un criterio di arresto: per ogni campione (x_k, y_k) calcolare $\hat{y}_k$ (forward), calcolare g_j (5.10), calcolare e_h (5.15), aggiornare w_{hj} , v_{ih} , θ_j , γ_h con (5.11)–(5.14).

BP standard vs BP accumulato

- **BP standard:** aggiorna i parametri dopo **ogni singolo campione**; aggiornamenti frequenti ma "rumorosi" (gli aggiornamenti di campioni diversi possono parzialmente annullarsi), richiede più iterazioni ma spesso converge più velocemente in pratica su dataset grandi.
- **BP accumulato (batch):** minimizza l'errore accumulato su **tutto il training set** prima di aggiornare; aggiornamenti meno frequenti ma più stabili; può risultare più lento quando il training set è molto grande.

Capacità rappresentativa e overfitting

Una rete feed-forward con un solo strato nascosto, con un numero sufficiente di neuroni, può approssimare **qualsunque funzione continua con accuratezza arbitraria** [Hornik et al., 1989] (teorema di approssimazione universale). Tuttavia il numero ottimo di neuroni nascosti non ha una formula chiusa e si sceglie per tentativi (trial-and-error). Le reti BP soffrono facilmente di **overfitting** (errore di training che scende mentre l'errore di test/validazione sale). Due strategie principali per mitigarlo: - **Early stopping:** si interrompe l'addestramento non appena l'errore di validazione inizia a salire, pur continuando a scendere quello di training. - **Regolarizzazione:** si aggiunge alla funzione obiettivo un termine che penalizza la complessità della rete (tipicamente la norma dei pesi), scoraggiando pesi troppo grandi.

Capitolo 7 — Bayes Classifiers (Classificatori Bayesiani)

Bayes Decision Theory (teoria delle decisioni bayesiana)

L'idea di base è **classificare in modo ottimo sotto incertezza**, minimizzando un costo atteso invece di massimizzare una ricompensa attesa. Dati N classi $y = \{c_1, \dots, c_N\}$ e un costo λ_{ij} (costo di classificare come c_i un esempio che in realtà è c_j), il **rischio condizionato** di assegnare $\mathbf{x}$ alla classe c_i è

$$R(c_i | \mathbf{x}) = \sum_{j=1}^N \lambda_{ij} P(c_j | \mathbf{x})$$

cioè il costo atteso di classificare $\mathbf{x}$ come c_i , mediato su tutte le classi vere possibili pesate dalla probabilità a posteriori. Il **rischio complessivo** di un classificatore h è $R(h) = \mathbb{E}_{\mathbf{x}}[R(h(\mathbf{x}) | \mathbf{x})]$, ed è minimizzato scegliendo per ogni $\mathbf{x}$ la classe che minimizza il rischio condizionato:

$$h^*(x) = \operatorname{argmin}_{c \in y} R(c | x)$$

h^* è il **classificatore Bayes ottimo** e il rischio associato $R(h^*)$ è il **rischio di Bayes**: rappresenta il limite teorico superiore di accuratezza raggiungibile da qualunque modello ($1 - R(h^*)$ è la miglior accuratezza possibile).

Se l'obiettivo è semplicemente minimizzare il tasso di errore (costo 0/1: $\lambda_{ij} = 0$ se $i = j$, 1 altrimenti), il rischio condizionato diventa $R(c | \mathbf{x}) = 1 - P(c | \mathbf{x})$, e il classificatore ottimo si riduce a scegliere la classe con **massima probabilità a posteriori**:

$$h^*(x) = \operatorname{argmax}_{c \in y} P(c | x)$$

Il problema pratico è che $P(c | \mathbf{x})$ è spesso difficile da stimare direttamente dai dati finiti. Esistono due strategie: i **modelli discriminativi** (es. alberi di decisione, reti neurali, SVM) stimano $P(c | \mathbf{x})$ direttamente; i **modelli generativi** stimano prima la probabilità congiunta $P(\mathbf{x}, c)$ e ne derivano $P(c | \mathbf{x}) = P(\mathbf{x}, c) / P(\mathbf{x})$. Applicando il teorema di Bayes:

$$P(c | \mathbf{x}) = \frac{P(c) P(\mathbf{x} | c)}{P(\mathbf{x})}$$

dove $P(c)$ è la **probabilità a priori** (proporzione di ciascuna classe, stimabile dalla frequenza nel training set), $P(\mathbf{x} | c)$ è la **probabilità di classe condizionata** (o **likelihood**) del campione rispetto alla classe, e $P(\mathbf{x})$

è l'**evidenza**, costante rispetto a c e quindi ignorabile nel confronto tra classi. Tutti i classificatori bayesiani generativi (Naive Bayes incluso) ruotano attorno alla stima di $P(c)$ e $P(\mathbf{x} | c)$.

Stima di Massima Verosimiglianza (MLE)

Una strategia generale per stimare $P(\mathbf{x} | c)$ è ipotizzare una forma parametrica fissa (es. Gaussiana con parametri $\theta_c = (\mu_c, \sigma_c^2)$) e stimarne i parametri dai dati. Dato D_c (i campioni della classe c , assunti i.i.d.), la **verosimiglianza** dei dati dato θ_c è

$$P(D_c | \theta_c) = \prod_{\mathbf{x} \in D_c} P(\mathbf{x} | \theta_c)$$

Poiché il prodotto di molti termini piccoli causa underflow numerico, si usa la **log-verosimiglianza** $LL(\theta_c) = \sum_{\mathbf{x} \in D_c} \log P(\mathbf{x} | \theta_c)$, e la stima MLE è quella che la massimizza: $\hat{\theta}_c = \operatorname{argmax}_{\theta_c} LL(\theta_c)$. Intuitivamente, si cerca il parametro che rende i dati osservati "più probabili possibile". Per dati continui con distribuzione Gaussiana $p(\mathbf{x} | c) \sim N(\mu_c, \sigma_c^2)$, la soluzione in forma chiusa è

$$\hat{\mu}_c = \frac{1}{|D_c|} \sum_{\mathbf{x} \in D_c} \mathbf{x} \quad \hat{\sigma}_c^2 = \frac{1}{|D_c|} \sum_{\mathbf{x} \in D_c} (\mathbf{x} - \hat{\mu}_c)(\mathbf{x} - \hat{\mu}_c)^\top$$

cioè media e varianza campionarie: risultato intuitivo, ma l'accuratezza dipende fortemente dal fatto che la forma di distribuzione ipotizzata rispecchi davvero i dati reali.

Naive Bayes Classifier

Il problema di stimare $P(\mathbf{x} | c)$ è che è una probabilità congiunta su **tutti** gli attributi: con d attributi binari servirebbero 2^d combinazioni, tipicamente $\gg$ del numero di esempi disponibili m (stima infattibile con dati finiti). Il Naive Bayes risolve il problema con l'**assunzione di indipendenza condizionata degli attributi**: data la classe, tutti gli attributi sono assunti indipendenti tra loro. Questo permette di fattorizzare:

$$P(c | \mathbf{x}) = \frac{P(c)P(\mathbf{x} | c)}{P(\mathbf{x})} = \frac{P(c)}{P(\mathbf{x})} \prod_{i=1}^d P(x_i | c)$$

Poiché $P(\mathbf{x})$ è uguale per tutte le classi, il classificatore finale è

$$h_{nb}(\mathbf{x}) = \operatorname{argmax}_{c \in \mathcal{Y}} P(c) \prod_{i=1}^d P(x_i | c)$$

Addestramento: si stima $P(c)$ come frequenza relativa della classe, $P(c) = |D_c|/|D|$, e per ogni attributo discreto $P(x_i | c) = |D_{c,x_i}|/|D_c|$ (frazione dei campioni di classe c che hanno il valore x_i sull'attributo i); per attributi continui si usa tipicamente una Gaussiana $P(x_i | c) = \frac{1}{\sqrt{2\pi}\sigma_{c,i}} \exp\left(-\frac{(x_i - \mu_{c,i})^2}{2\sigma_{c,i}^2}\right)$, con media e varianza stimate come nella MLE sopra. Esempio concettuale (dataset Watermelon): per stimare $P(\text{color} = \text{green} | \text{ripe} = \text{true})$ si contano quanti esempi con classe "true" hanno color=green e si divide per il numero totale di esempi in classe "true" — esattamente $|D_{c,x_i}|/|D_c|$.

Correzione di Laplace: se un valore di attributo non compare mai insieme a una classe nel training set, la stima diventa 0, e per il prodotto ciò azzerava l'intera posteriore indipendentemente dagli altri attributi

(comportamento indesiderato). Per evitarlo, con N = numero di classi distinte e N_i = numero di valori distinti del i -esimo attributo:

$$\hat{P}(c) = \frac{|D_c| + 1}{|D| + N} \quad \hat{P}(x_i | c) = \frac{|D_{c,x_i}| + 1}{|D_c| + N_i}$$

Questo è essenzialmente uno smoothing additivo (prior uniforme) che evita probabilità zero mantenendo comunque le stime vicine a quelle empiriche per $|D|$ grande.

Semi-Naive Bayes (cenni)

Poiché l'indipendenza totale è spesso irrealistica, i classificatori **semi-naive Bayes** rilassano parzialmente l'assunzione permettendo a ciascun attributo di dipendere al più da **un altro attributo** oltre alla classe (**One-Dependent Estimator**, ODE): $P(c | x) \propto P(c) \prod_{i=1}^d P(x_i | c, pa_i)$, dove pa_i è l'attributo "genitore" di x_i . Varianti principali: **SPODE** (tutti gli attributi condividono lo stesso genitore, scelto ad es. per cross-validation); **TAN** (Tree Augmented Naive Bayes), che costruisce un albero di dipendenze massimizzando la somma delle informazioni mutue condizionate $I(x_i, x_j | y)$ tra coppie di attributi (albero di copertura a peso massimo); **AODE**, che integra (in ensemble) più modelli SPODE, uno per ogni possibile scelta di super-genitore.

Reti Bayesiane

Una **rete bayesiana** $B = \langle G, \Theta \rangle$ usa un **grafo diretto aciclico (DAG)** G per rappresentare le dipendenze condizionate tra attributi, più delle **tabelle di probabilità condizionata (CPT)** Θ per quantificarle. L'assunzione strutturale chiave: dato l'insieme dei genitori π_i , ogni nodo x_i è indipendente da tutti i suoi nodi non discendenti. Questo permette di fattorizzare la probabilità congiunta come prodotto di fattori locali:

$$P_B(x_1, \dots, x_d) = \prod_{i=1}^d P_B(x_i | \pi_i) = \prod_{i=1}^d \theta_{x_i | \pi_i}$$

cioè, invece di specificare l'intera tabella congiunta (esponenziale), basta specificare, per ogni nodo, la sua CPT rispetto ai soli genitori. La struttura codifica tre pattern locali di dipendenza tra tre variabili: **genitore comune** (x_3, x_4 figli di x_1 : indipendenti se si osserva x_1), **V-structure/collider** (x_4 figlio di x_1, x_2 : x_1, x_2 diventano dipendenti se si osserva x_4), **cascata/catena** ($x \rightarrow y \rightarrow z$: indipendenza data la variabile intermedia). Il criterio generale per leggere le indipendenze condizionate dal grafo è la **d-separazione** (si costruisce il **grafo morale** "sposando" i genitori con figlio comune e rendendo tutti gli archi non orientati).

Apprendimento della struttura: si usa una funzione di score basata sul principio **MDL (Minimum Description Length)**, che bilancia complessità del modello ed errore sui dati: $S(B | D) = f(\theta)|B| - LL(B | D)$, dove $|B|$ è il numero di parametri della rete, $f(\theta)$ è il costo di codifica per parametro, e $LL(B | D) = \sum_{i=1}^m \log P_B(x_i)$ è la log-verosimiglianza della rete sui dati: si cerca la rete che minimizza lo score (modello più semplice che spiega bene i dati, in linea col rasoio di Occam).

Inferenza: dato un'evidenza (valori osservati), si vuole calcolare la probabilità a posteriori di una feature interrogata; l'inferenza esatta è spesso costosa, quindi si ricorre a metodi approssimati come il **Gibbs sampling**: partendo da un campione casuale, si generano nuovi campioni modificando iterativamente le variabili non-evidenza (formando una catena di Markov); se su T campionamenti n_q sono coerenti con la query q , la posteriore si approssima come $P(Q = q | E = e) \simeq n_q/T$.

EM Algorithm (Expectation-Maximization)

L'algoritmo **EM** [Dempster et al., 1977] serve a stimare i parametri di un modello quando i dati di training sono **incompleti**, cioè esistono **variabili latenti** non osservate (es. la feature "root" mancante in alcuni esempi del dataset watermelon). Siano $\mathbf{X}$ le variabili osservate, $\mathbf{Z}$ le variabili latenti, Θ i parametri del modello. La stima MLE massimizzerebbe $LL(\Theta | \mathbf{X}, \mathbf{Z}) = \ln P(\mathbf{X}, \mathbf{Z} | \Theta)$, ma non è risolvibile direttamente perché $\mathbf{Z}$ è ignoto; si usa invece la log-verosimiglianza marginale dei soli dati osservati:

$$LL(\Theta | \mathbf{X}) = \ln P(\mathbf{X} | \Theta) = \ln \sum_{\mathbf{Z}} P(\mathbf{X}, \mathbf{Z} | \Theta)$$

L'idea di EM è iterare due passi finché non si converge (a un ottimo locale):

- **E-step (Expectation):** dato lo stato corrente dei parametri Θ^t , si inferisce la distribuzione delle variabili latenti $P(\mathbf{Z} | \mathbf{X}, \Theta^t)$ e si calcola il valore atteso della log-verosimiglianza completa rispetto a $\mathbf{Z}$:

$$Q(\Theta | \Theta^t) = \mathbb{E}_{\mathbf{Z} | \mathbf{X}, \Theta^t} LL(\Theta | \mathbf{X}, \mathbf{Z})$$

- **M-step (Maximization):** si aggiornano i parametri massimizzando questo valore atteso:

$$\Theta^{t+1} = \underset{\Theta}{\operatorname{argmax}} Q(\Theta | \Theta^t)$$

In sostanza: nell'E-step si "riempiono" probabilisticamente i dati mancanti usando i parametri correnti (invece di un singolo valore, si usa l'intera distribuzione/aspettativa di $\mathbf{Z}$); nell'M-step si ri-stima Θ con MLE standard trattando quella distribuzione come se fosse osservata. Iterando E-step e M-step la log-verosimiglianza osservata non decresce mai, e il procedimento converge a un massimo (locale) — è lo schema alla base, ad esempio, della stima dei parametri in modelli a mistura gaussiana o di reti bayesiane con dati mancanti.

Capitolo 9 — Clustering

Task del Clustering

Il clustering è il compito principale dell'**apprendimento non supervisionato**: dato un dataset $D = \{x_1, \dots, x_m\}$ di campioni **non etichettati** (ogni x_i è un vettore n -dimensionale), un algoritmo di clustering partiziona D in k cluster disgiunti $C = \{C_1, \dots, C_k\}$, con $C_i \cap C_j = \emptyset$ per $i \neq j$ e $\bigcup_i C_i = D$. Ad ogni campione viene associata un'etichetta di cluster $\lambda_j \in \{1, \dots, k\}$, e il risultato si rappresenta come vettore $\lambda = (\lambda_1, \dots, \lambda_m)$. Il clustering serve sia come tecnica esplorativa (scoprire la struttura intrinseca dei dati) sia come pre-processing per altri task (es. classificazione).

Misure di Validità (Performance Measure)

Un buon clustering deve avere **alta similarità intra-cluster** e **bassa similarità inter-cluster**. Le misure si dividono in **indici esterni** (confrontano il risultato con un modello di riferimento/etichette vere) e **indici interni** (valutano il clustering senza riferimenti esterni, es. coesione/separazione).

Indici esterni. Per ogni coppia di campioni (x_i, x_j) , con $i < j$, si definiscono quattro contatori confrontando l'assegnazione stimata λ con quella vera λ^* :

$$a = |SS|, SS = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*\}$$

$$b = |SD|, SD = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*\}$$

$$c = |DS|, DS = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*\}$$

$$d = |DD|, DD = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*\}$$

a conta le coppie messe correttamente nello stesso cluster, d quelle correttamente separate; b, c sono gli errori. Da qui si calcolano tre indici, tutti in $[0, 1]$ (maggiore = migliore):

$$JC = \frac{a}{a + b + c} \quad FMI = \sqrt{\frac{a}{a + b} \cdot \frac{a}{a + c}} \quad RI = \frac{2(a + d)}{m(m - 1)}$$

dove m è il numero totale di campioni (quindi $m(m - 1)/2$ è il totale delle coppie).

Indici interni: coesione e separazione. La coesione (quanto sono vicini i punti dentro un cluster) si misura con la *Within-cluster Sum of Squares*:

$$WSS = \sum_i \sum_{x \in C_i} (x - m_i)^2 \quad (\text{più piccolo è meglio})$$

la separazione (quanto i cluster sono distanti tra loro) con la *Between-cluster Sum of Squares*:

$$BSS = \sum_i |C_i| (m_g - m_i)^2 \quad (\text{più grande è meglio})$$

dove m_i è la media del cluster i e m_g è la media globale del dataset. Vale $WSS + BSS = SSE_{tot}$ (costante), da cui l'indice combinato $R^2 = 1 - WSS/SSE$ (più grande è meglio): rappresenta la proporzione di varianza totale spiegata dal clustering.

Altre quattro grandezze di base per gli indici interni: $avg(C)$ (distanza media tra i punti dentro C), $diam(C)$ (distanza massima dentro C), $d_{min}(C_i, C_j)$ (distanza minima tra due cluster) e $d_{cen}(C_i, C_j)$ (distanza tra i centroidi). Si combinano in:

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{avg(C_i) + avg(C_j)}{d_{cen}(C_i, C_j)} \right) \quad (\text{piccolo è meglio})$$

$$DI = \min_{1 \leq i \leq k} \left\{ \min_{j \neq i} \left(\frac{d_{min}(C_i, C_j)}{\max_l diam(C_l)} \right) \right\} \quad (\text{grande è meglio})$$

Il **Davies-Bouldin Index (DBI)** penalizza cluster larghi e vicini tra loro; il **Dunn Index (DI)** premia cluster compatti e ben separati (rapporto tra la minima distanza inter-cluster e il massimo diametro intra-cluster).

Misure di Distanza

Una funzione distanza deve soddisfare quattro assiomi: **non-negatività** ($dist \geq 0$), **identità degli indiscernibili** ($dist(x_i, x_j) = 0 \iff x_i = x_j$), **simmetria** ($dist(x_i, x_j) = dist(x_j, x_i)$) e **subadditività/triangolare** ($dist(x_i, x_j) \leq dist(x_i, x_k) + dist(x_k, x_j)$). La misura più usata per attributi continui è la **distanza di Minkowski**:

$$dist(x_i, x_j) = \left(\sum_{u=1}^n |x_{iu} - x_{ju}|^p \right)^{1/p}$$

con $p = 2$ si ottiene la distanza **Euclidea**, con $p = 1$ la distanza di **Manhattan**. Per attributi categorici *non-ordinali* (nominali, senza un ordine naturale) si usa la **Value Difference Metric (VDM)**:

$$VDM_p(a, b) = \sum_{i=1}^k \left| \frac{m_{u,a,i}}{m_{u,a}} - \frac{m_{u,b,i}}{m_{u,b}} \right|^p$$

dove $m_{u,a}$ è il numero di campioni con valore a sull'attributo u , e $m_{u,a,i}$ è il numero di quelli con valore a che appartengono al cluster i : due valori categorici sono vicini se producono distribuzioni simili tra i cluster. Per dataset con attributi misti si combina tutto in **MinkovDM_p**, sommando la parte Minkowski (attributi continui) e la parte VDM (attributi categorici) sotto la stessa radice p -esima; se gli attributi hanno importanza diversa si introduce una **distanza pesata** con pesi $\omega_i \geq 0$, $\sum_i \omega_i = 1$.

K-Means Clustering

È l'algoritmo di **clustering basato su prototipi** più noto: assume che la struttura dei cluster sia rappresentabile da k vettori medi (centroidi) $\mu_1, \dots, \mu_k$, e minimizza l'errore quadratico:

$$E = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

E misura la vicinanza tra i punti e il centroide del proprio cluster: minimizzarlo significa massimizzare la coesione interna. Poiché la minimizzazione esatta è NP-hard, si usa una procedura **iterativa** (Algoritmo 9.1):
 1. **Inizializzazione**: scegli casualmente k campioni come centroidi iniziali $\mu_1, \dots, \mu_k$.
 2. **Assegnazione**: per ogni x_j , calcola la distanza $d_{ji} = \|x_j - \mu_i\|_2$ da ciascun centroide e assegna x_j al cluster più vicino:

$\lambda_j = \arg \min_i d_{ji}$. 3. **Aggiornamento**: ricalcola ogni centroide come media dei punti assegnati: $\mu'_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$. 4. Ripeti i passi 2–3 finché nessun centroide cambia (convergenza).

L'algoritmo converge sempre (E decresce monotonamente) ma solo a un **minimo locale**, dipendente dall'inizializzazione; va inoltre specificato k a priori e funziona bene solo su cluster di forma "sferica" (dato che usa solo distanze dai centroidi, non informazione di densità).

Learning Vector Quantization (LVQ)

A differenza del clustering classico, LVQ assume che i dati **abbiano etichette** $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$ e usa questa informazione supervisionata per guidare l'apprendimento di q vettori prototipo $\{p_1, \dots, p_q\}$, ciascuno associato a un'etichetta iniziale t_i (Algoritmo 9.2). Ad ogni iterazione si estrae un campione (x_j, y_j) , si trova il prototipo più vicino p_{i^*} , e lo si aggiorna:

$$\text{se } y_j = t_{i^*} : p' = p_{i^*} + \eta \cdot (x_j - p_{i^*}) \quad \text{altrimenti: } p' = p_{i^*} - \eta \cdot (x_j - p_{i^*})$$

Se l'etichetta del campione coincide con quella del prototipo, il prototipo viene **spostato verso** il campione (la distanza si riduce di un fattore $(1 - \eta)$); altrimenti viene **allontanato**. Ogni prototipo p_i definisce una regione R_i (i punti più vicini a p_i che a qualsiasi altro prototipo), e l'insieme delle regioni forma una **tassellazione di Voronoi**: rappresentare i dati tramite i soli prototipi realizza una compressione lossy, da cui il nome "quantizzazione vettoriale".

Mixture-of-Gaussian Clustering

A differenza di k-means e LVQ, non usa prototipi ma un **modello probabilistico generativo**. Una gaussiana multivariata in n dimensioni ha densità

$$p(x) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

con μ vettore medio e Σ matrice di covarianza. Un **mixture-of-Gaussian** con k componenti è la combinazione convessa

$$p_M(x) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i), \quad \alpha_i > 0, \quad \sum_{i=1}^k \alpha_i = 1$$

dove α_i (coefficiente di mixture) è la probabilità a priori di scegliere la i -esima componente. Ogni cluster corrisponde a una componente gaussiana. L'ottimizzazione (simile a EM) alterna: calcolo della **probabilità a posteriori** (responsabilità) che x_j sia stato generato dalla componente i ,

$$\gamma_{ji} = \frac{\alpha_i \cdot p(x_j | \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l \cdot p(x_j | \mu_l, \Sigma_l)}$$

e aggiornamento dei parametri come medie pesate: $\mu'_i = \frac{\sum_j \gamma_{ji} x_j}{\sum_j \gamma_{ji}}$, $\Sigma'_i = \frac{\sum_j \gamma_{ji} (x_j - \mu'_i)(x_j - \mu'_i)^T}{\sum_j \gamma_{ji}}$, $\alpha'_i = \frac{1}{m} \sum_j \gamma_{ji}$. Si ripete fino a convergenza; infine ogni campione è assegnato al cluster con responsabilità massima: $\lambda_j = \arg \max_i \gamma_{ji}$ (Algoritmo 9.3). A differenza di k-means (assegnazione "hard"), qui l'assegnazione è "soft" (probabilistica) durante l'ottimizzazione, e i cluster possono avere forme ellissoidali (non solo sferiche) grazie a Σ_i .

Clustering Gerarchico — AGNES

L'**AGNES** (AGglomerative NESTing) è un algoritmo *bottom-up*: parte da m cluster singleton (un campione ciascuno) e a ogni round **fonde i due cluster più vicini**, ripetendo finché non si raggiunge il numero desiderato k di cluster (Algoritmo 9.5); il risultato è un **dendrogramma** che può essere "tagliato" a qualsiasi livello per ottenere il numero di cluster voluto, senza doverlo fissare a priori. La distanza tra due cluster può essere definita in tre modi:

$$d_{min}(C_i, C_j) = \min_{x \in C_i, z \in C_j} dist(x, z)$$

$$d_{max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} dist(x, z)$$

$$d_{avg}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{z \in C_j} dist(x, z)$$

(collegamento singolo, completo, medio). Ad ogni fusione si aggiorna una matrice di prossimità M : si tiene una riga/colonna per il cluster risultante e si elimina l'altra.

Clustering basato sulla densità — DBSCAN

DBSCAN assume che la struttura dei cluster sia determinata dalla **densità** locale dei punti, tramite due parametri $(\epsilon, MinPts)$: - **ϵ -neighborhood**: per $x_j \in D$, l'insieme dei punti a distanza $\leq \epsilon$ da x_j . - **Core object (oggetto core)**: x_j tale che $|N_\epsilon(x_j)| \geq MinPts$. - **Direttamente density-reachable**: x_j è direttamente raggiungibile da x_i se x_i è core e $x_j \in N_\epsilon(x_i)$. - **Density-reachable**: raggiungibilità tramite una catena di punti direttamente density-reachable. - **Density-connected**: x_i, x_j sono density-connected se esiste un punto o da cui entrambi sono density-reachable.

Un **cluster** è definito come il più grande sottoinsieme di punti density-connected (proprietà di *connectivity*) tale che ogni punto density-reachable da uno di essi appartiene al cluster (*maximality*). L'algoritmo (9.4) individua prima tutti gli oggetti core, poi espande i cluster a partire da un core object non ancora processato, propagandosi attraverso i punti direttamente density-reachable in coda; i punti che non sono né core né raggiungibili da alcun core restano **rumore/outlier**. A differenza di k-means, DBSCAN **non richiede di fissare k a priori**, trova cluster di **forma arbitraria** (non solo sferici) e identifica esplicitamente gli outlier. In pratica ϵ e $MinPts$ si stimano ordinando i punti per la loro *reachability distance* (distanza dal k -esimo vicino più prossimo, con $k = MinPts$) e cercando il "gomito" (knee) nel grafico ordinato: prima del gomito i punti sono core (bassa reachability distance), dopo sono rumore/bordo.

Capitolo 10 — Dimension Reduction and Metric Learning (Riduzione della Dimensionalità)

k-Nearest Neighbor Learning (kNN)

Il **k-NN** è un metodo di apprendimento supervisionato che non ha una vera fase di training: appartiene al **lazy learning** (memorizza solo i dati e agisce alla richiesta di predizione), a differenza dell'**eager learning** che costruisce un modello esplicito durante il training. Dato un campione di test, si trovano i k campioni di training più vicini (secondo una metrica di distanza) e si predice per **voto** (classificazione, eventualmente pesato: più vicino = peso maggiore) o per **media** (regressione). Il parametro k e la metrica di distanza scelta influenzano fortemente il risultato.

Errore di classificazione binaria del 1-NN: dato un test x e il suo vicino più prossimo z ,

$$P(err) = 1 - \sum_{c \in \mathcal{Y}} P(c|x)P(c|z)$$

Questa è la probabilità che x e z abbiano etichetta di classe diversa. Assumendo campionamento denso (per ogni x esiste sempre un campione a distanza δ arbitrariamente piccola) e detto $c^* = \arg \max_{c \in \mathcal{Y}} P(c|x)$ il classificatore Bayesiano ottimo, si dimostra che $P(err) \leq 2 \times (1 - P(c^*|x))$: **l'errore del 1-NN è al più il doppio dell'errore del classificatore Bayes-ottimo**, un risultato notevole per un metodo così semplice.

Curse of Dimensionality e Low-dimensional Embedding

L'assunzione di campionamento denso usata sopra è **impraticabile in alte dimensioni**: con $\delta = 0.001$ servono 1000 campioni per coprire densamente una singola feature, ma con 20 feature ne servirebbero $(10^3)^{20} = 10^{60}$. All'aumentare della dimensionalità i dati diventano sparsi e anche operazioni semplici come il prodotto scalare perdono di significato: questo è il **curse of dimensionality** (maledizione della dimensionalità).

Un rimedio generale è la **riduzione della dimensionalità**: trasformare lo spazio delle feature originale (alta dimensione) in un sottospazio a bassa dimensione. Questo è possibile perché spesso i dati osservati, pur essendo ad alta dimensione, contengono una struttura intrinseca (embedding) a dimensione molto più bassa, effettivamente rilevante per il task.

Multiple Dimensional Scaling (MDS)

MDS proietta i dati in uno spazio a dimensione ridotta **preservando le distanze fra i campioni**. Data la matrice delle distanze $D \in \mathbb{R}^{m \times m}$ nello spazio originale (con $dist_{ij}$ distanza fra x_i e x_j), si vuole trovare

rappresentazioni z_i tali che le distanze siano preservate. Si definisce la **matrice del prodotto interno (Gram matrix)** $B = Z^T Z$ dei campioni proiettati, con $b_{ij} = z_i^T z_j$. Assumendo dati centrati (media zero), si ottiene la relazione chiave che lega B a D :

$$b_{ij} = -\frac{1}{2} (dist_{ij}^2 - dist_{i.}^2 - dist_{.j}^2 + dist_{..}^2)$$

dove $dist_{i.}^2$, $dist_{.j}^2$ e $dist_{..}^2$ sono rispettivamente la media dei quadrati delle distanze sulla riga i , sulla colonna j e su tutta la matrice. Questa formula permette di **calcolare B direttamente da D** , senza conoscere le coordinate z_i .

Poi si esegue la **decomposizione agli autovalori** $B = V \Lambda V^T$ e si scelgono i $d' \ll d$ autovalori più grandi (matrice diagonale $\tilde{\Lambda}$) con i corrispondenti autovettori $\tilde{V}$, ottenendo le coordinate a bassa dimensione:

$$Z = \tilde{\Lambda}^{1/2} \tilde{V}^T \in \mathbb{R}^{d' \times m}$$

Ogni colonna di Z è la rappresentazione ridotta di un campione. In pratica basta preservare **approssimativamente** le distanze, quindi si scartano gli autovalori piccoli. **Algoritmo MDS**: (1) calcola $dist_{i.}^2, dist_{.j}^2, dist_{..}^2$; (2) calcola B ; (3) decomposizione agli autovalori di B ; (4) prendi i d' autovalori più grandi $\rightarrow$ output $\tilde{V} \tilde{\Lambda}^{1/2}$.

Un metodo più semplice di riduzione lineare consiste nell'applicare una trasformazione lineare $Z = W^T X$, con $W \in \mathbb{R}^{d \times d'}$ matrice di proiezione le cui colonne $w_1, \dots, w_{d'}$ formano una nuova base (ortogonale se $w_i^T w_j = 0$).

Principal Component Analysis (PCA)

PCA è il metodo di riduzione lineare più importante: cerca un iperpiano su cui proiettare i dati con due proprietà equivalenti: 1. **Minimo errore di ricostruzione**: i campioni proiettati e ricostruiti devono restare vicini agli originali; 2. **Massima varianza**: le proiezioni dei campioni sull'iperpiano devono essere il più distanti possibile tra loro.

Entrambe le derivazioni portano allo stesso problema di ottimizzazione (con dati centrati, $W^T W = I$ ortonormale):

$$\max_W \text{tr}(W^T X X^T W) \quad \text{s.t.} \quad W^T W = I$$

$X X^T$ è (proporzionale al)la **matrice di covarianza** dei dati centrati. Massimizzare questa traccia significa massimizzare la varianza totale dei dati proiettati lungo le direzioni date dalle colonne di W .

Risolvendo con **moltiplicatori di Lagrange** si ottiene un problema di autovalori:

$$X X^T w = \lambda w \quad (\text{eq. 10.17})$$

cioè w deve essere un **autovettore** della matrice di covarianza $X X^T$ con autovalore λ . Si esegue la decomposizione agli autovalori di $X X^T$, si ordinano $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{d'}$, e la soluzione ottima è data dai d' autovettori con autovalore più grande: $W^* = (w_1, \dots, w_{d'})$.

Algoritmo PCA: (1) centra i dati sottraendo la media; (2) calcola la matrice di covarianza $X X^T$; (3) decomposizione agli autovalori; (4) prendi i d' autovettori con autovalore maggiore $\rightarrow$ output W^* . La proiezione di un nuovo campione si ottiene con $z = W^{*T}(x - \bar{x})$.

La dimensione d' si sceglie per validazione incrociata oppure fissando una soglia t (es. 95%) sulla **varianza spiegata** cumulata trattenuta. Ridurre la dimensionalità comporta perdita di informazione ma rende il campionamento più denso e **riduce il rumore**, poiché gli autovettori con autovalori più piccoli spesso catturano rumore piuttosto che segnale.

Kernelized PCA (KPCA)

Se la struttura è non lineare, PCA lineare fallisce (es. dati su una superficie a S in 3D). **KPCA** applica il kernel trick alla PCA: si suppone $z_i = \phi(x_i)$ una mappatura non lineare esplicita, ma poiché ϕ è generalmente ignota si introduce la funzione kernel $\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$. Sostituendo $w_j = \sum_i \phi(x_i) \alpha_i^j$ nel problema agli autovalori originale si ottiene:

$$K \alpha^j = \lambda_j \alpha^j$$

dove K è la **matrice kernel** ($(K)_{ij} = \kappa(x_i, x_j)$). Si tratta ancora di un problema agli autovalori, ma su K (dimensione $m \times m$) invece che sulla covarianza: si prendono gli autovettori α^j con i d' autovalori maggiori. Per un nuovo campione x , la coordinata proiettata sulla j -esima dimensione è:

$$z_j = \sum_{i=1}^m \alpha_i^j \kappa(x_i, x)$$

Il calcolo somma su **tutti** i campioni di training, rendendo KPCA costoso computazionalmente rispetto alla PCA lineare.

Manifold Learning

Un **manifold (varietà)** è uno spazio topologico localmente omeomorfo allo spazio Euclideo: localmente si comporta come uno spazio Euclideo (si può usare la distanza euclidea), anche se globalmente è complesso e non lineare. L'idea è definire mappature di riduzione **localmente** e poi estenderle globalmente.

Isomap sostituisce la distanza euclidea (che può essere fuorviante su un manifold curvo) con la **distanza geodetica** (il cammino più breve lungo la varietà). Si costruisce un grafo di vicinato (ogni punto connesso solo ai suoi k vicini più prossimi secondo distanza euclidea), si calcola la distanza geodetica come cammino minimo nel grafo (algoritmo di **Dijkstra** o **Floyd**), e infine si applica **MDS** su questa matrice di distanze geodetiche per ottenere l'embedding a bassa dimensione.

Locally Linear Embedding (LLE) cerca invece di preservare le relazioni **lineari locali** fra vicini. Per ogni campione x_i si trova l'insieme dei vicini Q_i e si calcolano i pesi di ricostruzione w_{ij} risolvendo:

$$\min_{w_1, \dots, w_m} \sum_{i=1}^m \left\| x_i - \sum_{j \in Q_i} w_{ij} x_j \right\|_2^2 \quad \text{s.t.} \quad \sum_{j \in Q_i} w_{ij} = 1$$

cioè x_i è ricostruito come combinazione lineare (a coefficienti che sommano a 1) dei suoi vicini. Poi si cercano le coordinate a bassa dimensione z_i che **preservano gli stessi pesi** w_{ij} :

$$\min_{z_1, \dots, z_m} \sum_{i=1}^m \left\| z_i - \sum_{j \in Q_i} w_{ij} z_j \right\|_2^2$$

In forma matriciale, con $M = (I - W)^T (I - W)$, il problema diventa $\min_Z \text{tr}(Z M Z^T)$ s.t. $Z Z^T = I$, risolto prendendo gli autovettori di M corrispondenti ai d' **autovalori più piccoli** (a differenza di PCA, qui si vogliono i più piccoli perché si minimizza un errore di ricostruzione).

Metric Learning

L'idea di fondo: poiché ogni spazio corrisponde a una metrica di distanza, cercare uno spazio a bassa dimensione buono equivale a **imparare direttamente una metrica di distanza** appropriata, piuttosto che

imparare prima una trasformazione. Partendo dalla distanza euclidea pesata per feature $\text{dist}_{wed}^2(x_i, x_j) = (x_i - x_j)^T W (x_i - x_j)$ con W diagonale (pesi $w_k \geq 0$), si generalizza sostituendo W con una matrice **simmetrica semidefinita positiva** M , ottenendo la **distanza di Mahalanobis**:

$$\text{dist}_{mah}^2(x_i, x_j) = (x_i - x_j)^T M (x_i - x_j) = \|x_i - x_j\|_M^2$$

M (semidefinita positiva, cioè $M = PP^T$ per qualche matrice P) cattura anche correlazioni tra feature (non solo assi ortogonali pesati). **Metric learning** significa imparare M dai dati, tipicamente ottimizzando le prestazioni di un classificatore nearest-neighbor.

Neighbourhood Component Analysis (NCA) sostituisce il voto binario del kNN con un **voto probabilistico**: la probabilità che x_j “influenzi” la classificazione di x_i è

$$p_{ij} = \frac{\exp(-\|x_i - x_j\|_M^2)}{\sum_l \exp(-\|x_i - x_l\|_M^2)}$$

(più x_j è vicino a x_i secondo M , maggiore il peso). L'accuratezza leave-one-out (LOO) attesa per x_i è $p_i = \sum_{j \in \Omega_i} p_{ij}$ (somma sui vicini Ω_i della stessa classe), e NCA massimizza l'accuratezza LOO totale sull'intero dataset (equivalentemente minimizza $1 - \sum_i \sum_{j \in \Omega_i} p_{ij}$) rispetto a P , dove $M = PP^T$.

In alternativa si può inserire **conoscenza di dominio**: dati un insieme di vincoli *must-link* $\mathcal{M}$ (coppie simili) e *cannot-link* $\mathcal{C}$ (coppie dissimili),

$$\min_M \sum_{(x_i, x_j) \in \mathcal{M}} \|x_i - x_j\|_M^2 \quad \text{s.t.} \quad \sum_{(x_i, x_k) \in \mathcal{C}} \|x_i - x_k\|_M \geq 1, \quad M \succeq 0$$

Si minimizza la distanza totale fra coppie simili mantenendo le coppie dissimili sufficientemente separate; $M \succeq 0$ garantisce che M resti semidefinita positiva (metrica valida). Infine, quando la M appresa è a **basso rango**, la sua decomposizione agli autovalori fornisce direttamente una matrice di proiezione P (con $d' < d$) utilizzabile per la riduzione della dimensionalità: metric learning e dimension reduction sono quindi due facce dello stesso problema.

Capitolo 11 — Feature Selection and Sparse Learning (Selezione delle Feature)

Introduzione

- **Feature rilevanti**: utili per il task di apprendimento. **Feature irrilevanti**: inutili. **Feature ridondanti**: la loro informazione è deducibile da altre feature (non trattate a fondo).
- **Feature selection**: selezionare/scartare le feature irrilevanti senza perdere quelle importanti. Serve a **alleviare la maledizione della dimensionalità** (curse of dimensionality) e a **ridurre la difficoltà di apprendimento**, mantenendo solo l'informazione chiave.
- Valutare *tutti* i possibili sottoinsiemi di feature è impraticabile (esplosione combinatoria). Si usa quindi un approccio iterativo: **genera** un sottoinsieme candidato → **valuta** la sua qualità → **genera il prossimo candidato** in base al risultato. Questo richiede due componenti: **subset search** (ricerca) e **subset evaluation** (valutazione).

Subset Search & Subset Evaluation

- **Forward search**: si parte dal sottoinsieme ottimo vuoto e si aggiunge, un passo alla volta, la feature che più migliora la valutazione (ricerca greedy).
- **Backward search**: si parte dall'insieme completo e si rimuovono via via le feature meno utili.
- **Bidirectional search**: aggiunge feature rilevanti e rimuove feature irrilevanti contemporaneamente.
- **Valutazione tramite entropia dell'informazione**: un sottoinsieme di feature A induce una partizione del dataset D in V sottoinsiemi D^v (uno per ogni combinazione di valori di A). Il **guadagno informativo** (information gain) del sottoinsieme è:

$$\text{Gain}(A) = \text{Ent}(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Ent}(D^v), \quad \text{Ent}(D) = - \sum_{k=1}^{|Y|} p_k \log_2 p_k$$

Qui $\text{Ent}(D)$ è l'entropia del dataset (quanto sono "mescolate" le classi, con p_k frazione della classe k), mentre $\text{Ent}(D^v)$ è l'entropia della sotto-partizione indotta dal valore v di A . Più il Gain è alto, più il sottoinsieme A riduce l'incertezza sulla label Y : la ricerca sceglie quindi ad ogni passo il sottoinsieme con Gain più alto. È l'idea alla base anche degli alberi decisionali.

Filter Methods — Relief

I **filter methods** selezionano le feature *senza* considerare il learner che verrà usato dopo (a differenza dei wrapper).

- **Relief** [Kira & Rendell, 1992] calcola per ogni feature una **statistica di rilevanza** δ^j ; le feature con δ^j sopra soglia (o le k con valore più alto) vengono selezionate.
- Per un campione x_i : **near-hit** $x_{i,nh}$ è il vicino più simile con la **stessa** label; **near-miss** $x_{i,nm}$ è il vicino più simile con label **diversa**.
- Statistica di rilevanza della feature j :

$$\delta^j = \sum_i -\text{diff}(x_i^j, x_{i,nh}^j)^2 + \text{diff}(x_i^j, x_{i,nm}^j)^2$$

dove $\text{diff}(x_a^j, x_b^j) = 0/1$ se j è discreta (0 se uguali, 1 altrimenti) oppure $|x_a^j - x_b^j|$ se j è continua (valori normalizzati in $[0, 1]$). **Intuizione:** δ^j è grande quando, sulla feature j , il near-hit è vicino (buon segno, stessa classe) e il near-miss è lontano (buon segno, classe diversa) — cioè la feature j separa bene le classi. - Il costo computazionale di Relief è **lineare** nel numero di campioni e di feature: molto efficiente. - **Relief-F** [Kononenko, 1994] estende Relief al multi-classe: si cerca un near-miss per *ogni* classe diversa dalla propria, pesato dalla proporzione p_l di quella classe nel dataset:

$$\delta^j = \sum_i -\text{diff}(x_i^j, x_{i,nh}^j)^2 + \sum_{l \neq k} (p_l \times \text{diff}(x_i^j, x_{i,l,nm}^j)^2)$$

Wrapper Methods — LVW

I **wrapper methods** usano direttamente le prestazioni del learner finale come metrica di valutazione del sottoinsieme: sono più mirati (feature “su misura” per il learner) e in genere più efficaci dei filter, ma molto più costosi (richiedono di riaddestrare il learner più volte).

- **LVW (Las Vegas Wrapper)** [Liu & Setiono, 1996]: applica una strategia **randomizzata** (Las Vegas) alla ricerca del sottoinsieme. Algoritmo: (1) genera un sottoinsieme di feature casuale; (2) stima l'errore del learner tramite **cross-validation** su quel sottoinsieme; (3) ripeti molte volte e tieni il sottoinsieme con errore minimo come soluzione finale. Nota: se il tempo è limitato, LVW può non restituire nulla.

Embedded Methods e Regularizzazione L1/L2

I **metodi embedded** uniscono selezione delle feature e addestramento del learner in un unico processo di ottimizzazione: le feature vengono selezionate automaticamente durante il training, tramite un termine di **regularizzazione**.

- **Ridge regression** [Tikhonov & Arsenin, 1977], regressione lineare con penalità **L2**:

$$\min_w \sum_{i=1}^m (y_i - w^\top x_i)^2 + \lambda \|w\|_2^2$$

- **LASSO** [Tibshirani, 1996], si ottiene sostituendo la norma L2 con la norma **L1**:

$$\min_w \sum_{i=1}^m (y_i - w^\top x_i)^2 + \lambda \|w\|_1$$

In entrambe, y_i è il target, x_i il vettore di feature, w i pesi da stimare e λ controlla la forza della regularizzazione. La differenza cruciale: **LASSO tende a produrre soluzioni sparse** (molti $w_j = 0$, cioè selezione automatica delle feature), mentre **Ridge riduce i pesi ma raramente li azzerà**. - **Perché L1 dà sparsità (intuizione**

geometrica): la soluzione è il punto di intersezione tra i contorni (curve di livello) dell'errore quadratico e i contorni della norma di regolarizzazione. I contorni della norma L1 formano un **rombo** con vertici sugli assi, mentre quelli della norma L2 formano un **cerchio**. Il rombo ha "punte" proprio sugli assi, quindi l'intersezione ottimale cade spesso su un vertice, dove una componente di w è **esattamente 0**; il cerchio, essendo liscio, non privilegia gli assi e quindi produce pesi piccoli ma non nulli. - **Soluzione via Proximal Gradient Descent (PGD)** [Boyd & Vandenberghe, 2004]: per risolvere $\min_x f(x) + \lambda \|x\|_1$ con f differenziabile e L-Lipschitz-continua nel gradiente ($\|\nabla f(x') - \nabla f(x)\|_2 \leq L\|x' - x\|_2$), si approssima f con un'espansione di Taylor del second'ordine, ottenendo ad ogni iterazione un passo di **gradient descent** $z^i = x_k - \frac{1}{L} \nabla f(x_k)$ seguito da una proiezione con soluzione in **forma chiusa** (soft-thresholding):

$$x_{k+1}^i = \begin{cases} z^i - \lambda/L, & \lambda/L < z^i \\ 0, & |z^i| \leq \lambda/L \\ z^i + \lambda/L, & z^i < -\lambda/L \end{cases}$$

Questa formula "spinge verso zero" ogni componente: se z^i è piccola (in modulo $\leq \lambda/L$) viene azzerata, altrimenti viene ridotta di λ/L verso zero — è il meccanismo esplicito che genera la sparsità del LASSO.

Sparse Representation e Dictionary Learning

- Una rappresentazione dei dati come matrice D è **sparsa** quando contiene molti zeri, ma non concentrati in righe/colonne intere. Vantaggi: rende molti problemi (es. classificazione testuale) **linearmente separabili** e riduce lo spazio di memorizzazione.
- **Dictionary learning**: dato un dataset $\{x_1, \dots, x_m\}$, si vuole imparare un **dizionario** $B \in \mathbb{R}^{d \times k}$ e, per ogni campione, una rappresentazione sparsa $\alpha_i \in \mathbb{R}^k$ (k = dimensione del vocabolario), risolvendo:

$$\min_{B, \alpha_i} \sum_{i=1}^m \|x_i - B\alpha_i\|_2^2 + \lambda \sum_{i=1}^m \|\alpha_i\|_1$$

Il primo termine misura quanto bene il dizionario ricostruisce ogni campione come combinazione $B\alpha_i$; il secondo, con la norma L1 (come in LASSO), forza le α_i a essere sparse. - Si risolve per **ottimizzazione alternata**: (1) fissato B , si trovano gli α_i (problema tipo LASSO); (2) fissati gli α_i (raccolti in A), si aggiorna B minimizzando $\|X - BA\|_F^2$ (norma di **Frobenius**, radice della somma dei quadrati di tutti gli elementi). **KSVD** [Aharon et al., 2006], generalizzazione del K-means, aggiorna B una colonna b_i alla volta tramite SVD, preservando la sparsità delle α .

Cenni: Compressed Sensing e Matrix Completion

- **Compressed sensing**: da un segnale campionato $y = \Phi x$ (con Φ matrice di misura, $n \ll m$ campioni) non si può in generale recuperare x ; ma se esiste una trasformata lineare Ψ (Fourier, coseno, wavelet...) tale che $x = \Psi s$ con s **sparsa**, e la matrice $A = \Phi\Psi$ soddisfa la **Restricted Isometry Property (RIP)** — $(1 - \delta_k)\|s\|_2^2 \leq \|A_k s\|_2^2 \leq (1 + \delta_k)\|s\|_2^2$ — allora s (e quindi x) può essere ricostruito quasi perfettamente. Si minimizza $\|s\|_0$ (numero di non-zeri, NP-hard) sostituendola con $\|s\|_1$ (equivalente sotto certe condizioni), risolvibile come un problema tipo LASSO.
- **Matrix completion** (es. sistemi di raccomandazione): recuperare una matrice X completa da osservazioni parziali A minimizzando $\text{rank}(X)$ sui valori noti; essendo NP-hard, si approssima con la **norma nucleare** $\|X\|_*$ (somma dei valori singolari), ottenendo un problema **convesso** risolvibile con Semi-Definite Programming.

Conclusione

Feature selection = **subset search** + **subset evaluation**, con tre famiglie di metodi: **filter** (es. Relief, indipendenti dal learner), **wrapper** (es. LVW, usano il learner come valutatore), **embedded** (regolarizzazione L1/L2, integrati nel training). La **sparse representation** (dictionary learning, compressed sensing, matrix completion) sfrutta la stessa idea della norma L1 per ottenere rappresentazioni compatte e ricostruzioni da dati parziali.

Capitolo 12 — Computational Learning Theory (Teoria dell'Apprendimento Computazionale)

Concetti Base

Si considera solo classificazione binaria, con campioni i.i.d. da una distribuzione $\mathcal{D}$. Un **concetto** c è la funzione ideale (ground-truth) che mappa istanze a etichette; l'insieme di tutti i concetti target che vogliamo apprendere è la **concept class** $\mathcal{C}$. L'algoritmo di apprendimento costruisce le sue ipotesi a partire da uno **spazio di ipotesi** $\mathcal{H}$, generalmente diverso da $\mathcal{C}$ perché $\mathcal{C}$ è sconosciuta.

$$E(h) = P_{x \sim \mathcal{D}}(h(x) \neq y)$$

è l'**errore di generalizzazione**: la probabilità che h sbagli su un'istanza estratta dalla distribuzione reale. Non è calcolabile direttamente (non conosciamo $\mathcal{D}$), quindi si usa il suo stimatore empirico

$$\hat{E}(h) = \frac{1}{m} \sum_{i=1}^m \mathbf{1}[h(x_i) \neq y_i]$$

cioè la frazione di errori su un training set di m campioni. Poiché il campionamento è i.i.d., $\hat{E}(h)$ è uno stimatore non distorto di $E(h)$ (stessa media), ma può discostarsene per la varianza dovuta al campione finito — è proprio questo scarto che tutto il capitolo cerca di limitare (bound). Un problema si dice **separabile/consistente** se $c \in \mathcal{H}$ (esiste un'ipotesi perfetta nello spazio considerato), **non-separabile** altrimenti ($c \notin \mathcal{H}$): in questo caso ogni ipotesi sbaglia su almeno un esempio, e diventa cruciale il concetto di **errore agnostico** (senza assumere che il target sia raggiungibile).

PAC Learning

L'idea centrale è che non serve imparare il concetto esatto: basta imparare un'ipotesi **quasi sempre quasi corretta**. Fissati un errore massimo tollerato ε e una confidenza $1 - \delta$:

$$P(E(h) \leq \varepsilon) \geq 1 - \delta$$

PAC Identify: l'algoritmo $\mathcal{L}$ identifica il concetto se, con probabilità almeno $1 - \delta$ (la parte "Probably"), produce un'ipotesi il cui errore è al più ε (la parte "Approximately Correct"). ε e δ sono due manopole indipendenti: ε controlla quanto l'ipotesi può essere imprecisa, δ controlla quanto spesso possiamo tollerare di "avere sfortuna" nel campionamento.

PAC Learnable: la concept class $\mathcal{C}$ è PAC-learnable rispetto a $\mathcal{H}$ se esiste $\mathcal{L}$ che, per ogni $0 < \varepsilon, \delta < 1$ e ogni distribuzione $\mathcal{D}$, riesce a fare PAC Identify usando un numero di campioni

$$m \geq \text{poly}(1/\varepsilon, 1/\delta, \text{size}(x), \text{size}(c))$$

polinomiale nei parametri del problema. Questo è il concetto chiave del capitolo: **learnability = esistenza di un algoritmo con sample complexity polinomiale** (non esponenziale) in $1/\varepsilon, 1/\delta$ e nella dimensione del problema. Se si richiede anche che il tempo di calcolo sia polinomiale si parla di **PAC Learning Algorithm** (efficiently PAC learnable). Assumendo che ogni campione richieda tempo costante, tempo e numero di campioni sono equivalenti, quindi ci si concentra sulla **sample complexity**: il più piccolo m che soddisfa il vincolo sopra. Il framework PAC serve a rispondere in modo teorico a domande pratiche: quanti dati servono per un buon modello, quali algoritmi imparano davvero, ecc.

Spazio di Ipotesi Finito ($|\mathcal{H}| < \infty$)

Caso separabile ($c \in \mathcal{H}$): la strategia naturale è eliminare tutte le ipotesi inconsistenti con il training set (sbagliano su almeno un esempio). Si dimostra che

$$m \geq \frac{1}{\varepsilon} \left(\ln |\mathcal{H}| + \ln \frac{1}{\delta} \right)$$

campioni bastano per garantire PAC identify: da qui **ogni spazio di ipotesi finito è PAC-learnable**, con errore di generalizzazione che converge a 0 al tasso $O(1/m)$. Intuitivamente, più $\mathcal{H}$ è grande (più ipotesi "false" possono sembrare consistenti per caso), più campioni servono, ma solo logicamente in $|\mathcal{H}|$.

Caso non-separabile ($c \notin \mathcal{H}$): qui si usa la **disuguaglianza di Hoeffding** (per variabili i.i.d. limitate in $[0, 1]$, la media empirica si concentra intorno al valore atteso con probabilità esponenziale in m). Applicandola si ottiene il risultato centrale della sezione:

Teorema 12.1. Per ogni $h \in \mathcal{H}$ finito e $0 < \delta < 1$:

$$P \left(|E(h) - \hat{E}(h)| \leq \sqrt{\frac{\ln |\mathcal{H}| + \ln(2/\delta)}{2m}} \right) \geq 1 - \delta$$

Il significato è fondamentale: **l'errore empirico è, con alta probabilità, una buona stima dell'errore di generalizzazione**, con uno scarto che si restringe al crescere di m (tasso $O(1/\sqrt{m})$) e cresce solo logicamente con la dimensione dello spazio di ipotesi. Questo giustifica formalmente il principio di **Empirical Risk Minimization (ERM)**: scegliere $h = \arg \min_{h' \in \mathcal{H}} \hat{E}(h')$, cioè minimizzare l'errore sul training set, è una strategia sensata perché $\hat{E}$ approssima bene E . Questo porta alla definizione di **Agnostic PAC Learnable**: $\mathcal{H}$ è agnostic PAC learnable se esiste $\mathcal{L}$ tale che $P(E(h) - \min_{h' \in \mathcal{H}} E(h') \leq \varepsilon) \geq 1 - \delta$, cioè l'ipotesi trovata è vicina alla *migliore possibile in $\mathcal{H}$* (non al target assoluto, che potrebbe non essere raggiungibile). Il Teorema 12.1 mostra che ogni $\mathcal{H}$ finito è agnostic PAC learnable.

VC Dimension

Negli spazi di ipotesi infiniti (intervalli reali, iperpiani, ecc.) non si può usare $|\mathcal{H}|$: serve una misura di complessità diversa. Dato un insieme di istanze $D = \{x_1, \dots, x_m\}$, ogni $h \in \mathcal{H}$ induce un vettore di etichette $(h(x_1), \dots, h(x_m))$, detto **dicotomia**. La **funzione di crescita** (growth function)

$$\Pi_{\mathcal{H}}(m) = \max_{\{x_1, \dots, x_m\} \subseteq \mathcal{X}} |\{(h(x_1), \dots, h(x_m)) \mid h \in \mathcal{H}\}|$$

conta il numero massimo di dicotomie distinte che $\mathcal{H}$ può generare su m punti (al più 2^m in classificazione binaria) e misura quindi il "potere rappresentativo" di $\mathcal{H}$. Se $\mathcal{H}$ realizza **tutte** le 2^m dicotomie su un insieme D di m punti si dice che $\mathcal{H}$ **shatters** (frantuma) D . La **VC dimension** è allora

$$VC(\mathcal{H}) = \max\{m : \Pi_{\mathcal{H}}(m) = 2^m\}$$

cioè la cardinalità del più grande insieme che $\mathcal{H}$ riesce a shatterare. Esempi canonici: gli intervalli chiusi $[a, b]$ su $\mathbb{R}$ hanno $VC = 2$ (si possono etichettare a piacere 2 punti, ma non 3, perché non si può ottenere il pattern $+, -, +$ con un solo intervallo); i separatori lineari nel piano $\mathbb{R}^2$ hanno $VC = 3$. In generale $VC = d$ significa: esiste un insieme di d punti shatterato, ma nessun insieme di $d + 1$ punti lo è.

Anche se $\mathcal{H}$ è infinito, se $VC(\mathcal{H}) = d$ è finito, la crescita di $\Pi_{\mathcal{H}}(m)$ è controllata dal **Lemma di Sauer**:

$$\Pi_{\mathcal{H}}(m) \leq \sum_{i=0}^d \binom{m}{i} \leq \left(\frac{em}{d}\right)^d$$

cioè da esponenziale la crescita diventa **polinomiale** in m (di grado d) non appena $m > d$: questo è il motivo per cui spazi infiniti con VC dimension finita restano comunque "imparabili". Sostituendo questo bound in una disuguaglianza tipo-Hoeffding generalizzata (Teorema 12.2) si arriva al risultato principale:

Teorema 12.3 (VC bound). Se $VC(\mathcal{H}) = d$, per ogni $m > d$, $h \in \mathcal{H}$, δ :

$$P\left(|E(h) - \hat{E}(h)| \leq \sqrt{\frac{8d \ln \frac{2em}{d} + 8 \ln \frac{4}{\delta}}{m}}\right) \geq 1 - \delta$$

Come nel caso finito, il bound converge a 0 (tasso $O(1/\sqrt{m})$), ma la caratteristica cruciale è che dipende **solo** da d e m : è **distribution-free** e **data-independent**, cioè vale per qualunque distribuzione $\mathcal{D}$ e qualunque dataset, il che lo rende molto generale ma anche piuttosto "lasco" (loose) nella pratica. Ne segue il **Teorema 12.4**: ogni $\mathcal{H}$ con VC dimension finita è (agnostic) PAC-learnable — VC dimension finita è quindi condizione sufficiente per la learnability, generalizzando il caso finito.

Rademacher Complexity

Il VC bound ignora completamente la distribuzione dei dati, quindi è spesso troppo pessimistico. La **Rademacher complexity** è un'alternativa che tiene conto della distribuzione: si introducono variabili **di Rademacher** $\sigma_i \in \{-1, +1\}$ i.i.d. con probabilità 0.5 ciascuna, che simulano "rumore casuale" sulle etichette. Se una funzione riesce a correlarsi bene anche con puro rumore, vuol dire che è molto flessibile (rischio di overfitting). Formalmente, la **Rademacher complexity empirica** di una classe di funzioni $\mathcal{F}$ su un campione $Z = \{z_1, \dots, z_m\}$ è

$$\hat{R}_Z(\mathcal{F}) = \mathbb{E}_{\sigma} \left[\sup_{f \in \mathcal{F}} \frac{1}{m} \sum_{i=1}^m \sigma_i f(z_i) \right]$$

e misura quanto bene, in media, la miglior $f \in \mathcal{F}$ riesce a "inseguire" un'etichettatura casuale: vale 0 se $\mathcal{F}$ contiene una sola funzione (nessuna flessibilità), tende a 1 quando $\mathcal{F}$ è così ricca da shatterare qualunque rumore. La versione **population** $R_m(\mathcal{F}) = \mathbb{E}_Z[\hat{R}_Z(\mathcal{F})]$ ne fa la media anche rispetto al campionamento del dataset da $\mathcal{D}$ — introducendo così la dipendenza dalla distribuzione che mancava nella VC dimension.

Teorema 12.6 (bound di generalizzazione per classificazione): per ogni $h \in \mathcal{H}$, con probabilità $\geq 1 - \delta$:

$$E(h) \leq \hat{E}(h) + R_m(\mathcal{H}) + \sqrt{\frac{\ln(1/\delta)}{2m}}$$

L'errore vero è limitato dall'errore empirico più un termine di complessità "su misura" per la distribuzione reale, quindi tipicamente **più stretto (tighter)** del VC bound. Il collegamento fra i due approcci è dato dal **Teorema 12.7**: $R_m(\mathcal{H}) \leq \sqrt{2 \ln \Pi_{\mathcal{H}}(m)/m}$, che mostra come il VC bound si possa **derivare come caso particolare** del bound basato su Rademacher complexity (combinandolo col Lemma di Sauer).

Stabilità

VC dimension e Rademacher complexity analizzano l'intero spazio $\mathcal{H}$, indipendentemente dall'algoritmo usato per scegliere h . L'analisi di **stabilità** guarda invece direttamente all'algoritmo $\mathcal{L}$: quanto cambia l'ipotesi prodotta se il training set cambia di poco? Si definiscono $D^{\setminus i}$ (dataset senza il campione i) e D^i (dataset con il campione i sostituito da un altro estratto i.i.d.). Una funzione di perdita $\ell(\mathcal{L}_D, z)$ misura l'errore dell'ipotesi appresa su D quando valutata sull'esempio z .

β -Uniform Stability: $\mathcal{L}$ soddisfa questa proprietà se, per ogni D , ogni i e ogni z ,

$$|\ell(\mathcal{L}_D, z) - \ell(\mathcal{L}_{D^{\setminus i}}, z)| \leq \beta$$

cioè rimuovere un singolo campione altera la perdita al più di β : l'algoritmo non "dipende troppo" da un singolo esempio. Se la loss è inoltre limitata da M , il **Teorema 12.8** dà

$$\ell(\mathcal{L}, \mathcal{D}) \leq \hat{\ell}(\mathcal{L}, D) + 2\beta + (4m\beta + M) \sqrt{\frac{\ln(1/\delta)}{2m}}$$

Quando $\beta = O(1/m)$ il tasso di convergenza torna a essere $O(1/\sqrt{m})$, coerente con VC e Rademacher bound: la stabilità è quindi un'altra via, complementare, per ottenere garanzie di generalizzazione, questa volta legate all'algoritmo specifico piuttosto che allo spazio di ipotesi nella sua interezza. Infine il **Teorema 12.9**: se un algoritmo che soddisfa il principio ERM è (uniformemente) stabile, allora lo spazio di ipotesi $\mathcal{H}$ è learnable — la stabilità dell'algoritmo implica la learnability, chiudendo il cerchio fra le diverse prospettive teoriche del capitolo (complessità dello spazio vs. comportamento dell'algoritmo).

Capitolo 13 — Semi-Supervised Learning (Apprendimento Semi-Supervisionato)

Concetti base e assunzioni

Nella pratica si dispone spesso di pochissimi campioni etichettati D_l (dimensione l) e di moltissimi campioni non etichettati D_u (dimensione u), con $l \ll u$, perché etichettare è costoso mentre raccogliere dati grezzi è economico. Due strategie sfruttano gli unlabeled in modo diverso: nell'**Active Learning** è il modello stesso a selezionare attivamente i campioni più informativi da sottoporre a un esperto per l'etichettatura; nel **Semi-Supervised Learning (SSL)** non c'è interazione con l'esperto, ma si sfrutta direttamente la distribuzione rivelata dai dati non etichettati. All'interno dell'SSL si distingue la **transductive learning** (il modello produce predizioni solo sugli unlabeled già visti in training, senza pretesa di generalizzare) dalla **pure SSL** (si apprende una funzione f generalizzabile anche a campioni futuri mai visti).

Per collegare la distribuzione empirica (rivelata dagli unlabeled) alle etichette servono ipotesi aggiuntive: - **Cluster assumption**: i dati presentano una struttura a cluster, e campioni nello stesso cluster condividono la stessa classe. - **Manifold assumption**: i campioni giacciono su una varietà (manifold) a bassa dimensionalità, e campioni vicini sulla varietà hanno output simili; è una generalizzazione della cluster assumption.

Queste assunzioni sono ciò che rende utile l'informazione non etichettata: senza di esse gli unlabeled non aggiungerebbero nulla al problema di apprendimento.

Metodi Generativi (Generative Methods / EM)

Si assume che ogni campione sia generato da una **mistura di Gaussiane (GMM)**, in cui ogni componente corrisponde esattamente a una classe:

$$p(x) = \sum_{i=1}^k \alpha_i \cdot p(x \mid \mu_i, \Sigma_i), \quad \alpha_i \geq 0, \quad \sum_{i=1}^k \alpha_i = 1$$

Il classificatore bayesiano ottimo diventa

$$f(x) = \arg \max_{j \in \mathcal{Y}} \sum_{i=1}^k p(y = j \mid \Theta = i, x) \cdot p(\Theta = i \mid x)$$

Il fattore $p(\Theta = i \mid x)$ (probabilità che x appartenga alla componente i) **non coinvolge l'etichetta**: può quindi essere stimato con maggiore accuratezza sfruttando la grande quantità di unlabeled disponibili, mentre solo $p(y = j \mid \Theta = i, x)$ richiede i labeled.

I parametri si stimano massimizzando la log-verosimiglianza su $D_l \cup D_u$ tramite **EM**: - **E-step**: per ogni unlabeled si calcola la responsabilità $\gamma_{ji} = \frac{\alpha_i p(x_j | \mu_i, \Sigma_i)}{\sum_{i=1}^k \alpha_i p(x_j | \mu_i, \Sigma_i)}$, cioè la probabilità a posteriori che il campione appartenga a ciascuna componente, dati i parametri correnti. - **M-step**: si aggiornano $\alpha_i, \mu_i, \Sigma_i$ usando sia gli unlabeled pesati per γ_{ji} sia i labeled della classe i (pesati 1), ad esempio $\mu_i = \frac{1}{\sum_j \gamma_{ji} + l_i} (\sum_{x_j \in D_u} \gamma_{ji} x_j + \sum_{(x_j, y_j) \in D_l \wedge y_j = i} x_j)$, dove l_i è il numero di labeled della classe i .

Questi metodi sono semplici, efficaci anche con etichette scarsissime, ma **dipendono fortemente dalla correttezza dell'assunzione generativa**: se il modello GMM non rispecchia la vera distribuzione, includere gli unlabeled può addirittura peggiorare le prestazioni.

Semi-Supervised SVM (S3VM)

Il principio è la **low-density separation**: l'iperpiano di separazione deve attraversare regioni a bassa densità di campioni, non tagliare un cluster a metà. Il modello più noto è la **Transductive SVM (TSVM, Joachims 1999)**:

$$\begin{aligned} \min_{w, b, \hat{y}, \xi} \quad & \frac{1}{2} \|w\|_2^2 + C_l \sum_{i=1}^l \xi_i + C_u \sum_{i=l+1}^m \xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i \quad (i \leq l), \quad \hat{y}_i(w^\top x_i + b) \geq 1 - \xi_i \quad (i > l), \quad \xi_i \geq 0 \end{aligned}$$

rispetto alla SVM classica si aggiungono le pseudo-etichette $\hat{y}_i$ degli unlabeled come variabili da ottimizzare, con un peso C_u separato da C_l per controllarne l'influenza. Il problema è combinatorio (non convesso), quindi TSVM usa una **ricerca locale iterativa**: addestra una SVM sui soli labeled, assegna pseudo-label agli unlabeled, poi scambia iterativamente le etichette di coppie di unlabeled con assegnazioni discordanti e verosimilmente errate, aumentando gradualmente C_u fino a raggiungere $C_u = C_l$ (così l'influenza degli unlabeled cresce man mano che ci si fida di più delle pseudo-label). Per il problema dello **sbilanciamento di classe** si divide C_u in C_u^+ e C_u^- , inizializzati come $C_u^+ = \frac{u_-}{u_+} C_u^-$ (dove u_-, u_+ sono il numero di unlabeled pseudo-etichettati negativi/positivi), per bilanciare l'impatto delle due classi. Poiché la ricerca su tutte le coppie è costosa, esistono varianti più efficienti come **LDS** (discesa del gradiente su graph kernel) e **meanS3VM**.

Graph-based SSL

Si costruisce un grafo $G = (V, E)$ i cui nodi sono i campioni di $D_l \cup D_u$ e i cui archi, pesati, riflettono la similarità, tipicamente tramite kernel Gaussiano:

$$W_{ij} = \exp\left(\frac{-\|x_i - x_j\|_2^2}{2\sigma^2}\right) \quad (i \neq j), \quad W_{ij} = 0 \text{ altrimenti}$$

L'idea è "propagare il colore" delle etichette dai nodi labeled a quelli unlabeled lungo il grafo. Si definisce una funzione **energia** che penalizza differenze di output tra nodi simili:

$$E(f) = \frac{1}{2} \sum_{i,j} W_{ij} (f(x_i) - f(x_j))^2 = f^\top (D - W) f$$

dove D è la matrice diagonale dei gradi. Partizionando $f = (f_l, f_u)$ e minimizzando rispetto a f_u (con f_l fissato ai valori noti) si ottiene la soluzione in forma chiusa (Zhu et al., 2003):

$$f_u = (D_{uu} - W_{uu})^{-1} W_{ul} f_l$$

valida per classificazione binaria. Per il caso **multiclasse** (Zhou et al., 2004) si usa una matrice di etichette F non-negativa, la matrice di propagazione normalizzata $S = D^{-1/2} W D^{-1/2}$, e l'aggiornamento iterativo

$$F(t+1) = \alpha S F(t) + (1 - \alpha) Y$$

che converge a $F^* = (1 - \alpha)(I - \alpha S)^{-1} Y$: a ogni passo un nodo eredita informazione dai vicini pesata da α e mantiene una quota $(1 - \alpha)$ della sua etichetta iniziale. Questo processo equivale a minimizzare un funzionale regolarizzato che bilancia smoothness sul grafo e fedeltà alle etichette note. I limiti principali sono il **costo in memoria/tempo** $O(n^2)$ (matrice W densa su tutti i campioni) e la natura essenzialmente transduttiva: un nuovo campione richiede di ricostruire il grafo o un meccanismo di predizione ad hoc.

Disagreement-based Methods (Co-training)

Questi metodi addestrano **più learner** e sfruttano il loro **disaccordo** per estrarre informazione dagli unlabeled. Il capostipite è il **Co-training** (Blum & Mitchell, 1998), pensato per dati multi-view: si assume che ogni campione sia descritto da due viste sufficienti e **condizionatamente indipendenti** date le classi (es. testo e immagini di una pagina web). Si addestra `model1` sulla `view1` e `model2` sulla `view2`; ciascun modello etichetta per l'altro i propri unlabeled più "confidenti", che vengono aggiunti al training set dell'altro. Un risultato teorico mostra che, se le viste sono sufficienti e indipendenti, le prestazioni di due weak learner possono essere **portate arbitrariamente in alto** sfruttando gli unlabeled. In pratica l'indipendenza condizionale raramente vale esattamente, quindi il guadagno è minore che in teoria, ma resta significativo anche sotto condizioni più deboli. Esistono varianti **single-view**, dove i learner differiscono per algoritmo, campionamento dei dati o iperparametri anziché per vista; teoricamente non serve il multi-view, basta un **disaccordo significativo** fra i learner perché lo scambio di pseudo-etichette migliori la generalizzazione (Zhou & Li, 2010).

Semi-Supervised Clustering

Nel clustering semi-supervisionato l'informazione di supervisione arriva in due forme: vincoli **must-link** (due campioni devono stare nello stesso cluster) e **cannot-link** (non devono stare nello stesso cluster), oppure un piccolo insieme di campioni etichettati.

Constrained k-means (Wagstaff et al., 2001) estende k-means assegnando ogni punto al cluster più vicino tra quelli **che non violano** alcun vincolo must-link/cannot-link; se il cluster più vicino viola un vincolo si prova il successivo più vicino, e se nessun cluster è ammissibile l'algoritmo restituisce un errore. È quindi k-means classico con un filtro di ammissibilità ad ogni assegnazione.

Constrained Seed k-means (Basu et al., 2002) usa invece i pochi campioni etichettati come **semi (seed)** per inizializzare i k centroidi (uno per classe/cluster); questi semi restano **fissi** nel proprio cluster per tutta l'esecuzione, mentre il resto dei punti viene assegnato e i centroidi aggiornati come nel k-means standard. In entrambi i casi la supervisione, anche minima, guida il clustering verso una partizione coerente con la conoscenza di dominio, riducendo l'ambiguità tipica del clustering puramente non supervisionato.

Capitolo 15 — Rule Learning (Apprendimento di Regole)

Concetti Base

Il **rule learning** è il ramo del machine learning che rappresenta la conoscenza appresa tramite **regole logiche simboliche**, del tipo “se ..., allora ...”, esplicite e interpretabili. Una regola logica ha la forma:

$$\oplus \leftarrow f_1 \wedge f_2 \wedge \dots \wedge f_L$$

dove $\oplus$ è la **head** (concetto target), $\leftarrow$ l’implicazione, e la **body** è la congiunzione (“AND”) di **letterali** f_i (atomi o loro negazione $\neg$ atomo). Si legge: “se letterale₁ e letterale₂ e ..., allora vale il target”. Un **rule set** può generare conflitti tra regole applicabili allo stesso esempio; si risolvono con **voting**, **ordering** (prima regola che matcha) o **meta-rule**.

Si distinguono due livelli di espressività: - **Regola proposizionale**: gli atomi sono condizioni fisse su attributi, es. $\text{ripe} \leftarrow (\text{root} = \text{curly}) \wedge (\text{umbilicus} = \text{hollow})$. Non può esprimere relazioni tra oggetti diversi. - **Regola del primo ordine**: usa costanti, **variabili**, **predicati/funzioni** di arità n , **termini**, **quantificatori** ($\forall, \exists$), permettendo di generalizzare su interi gruppi di individui, es. $\forall X (p(f(X)) \leftarrow p(X))$.

Sequential Covering

L’apprendimento di un **rule set** completo segue la strategia **Sequential Covering** (o **separate-and-conquer**): si impara una regola alla volta; ogni volta che una regola è appresa, si **rimuovono dal training set tutti i campioni coperti** da essa, e si ripete sui campioni rimanenti finché non restano (quasi) solo esempi di una classe.

Il problema chiave è la **single rule learning**: trovare l’insieme ottimale di letterali per il corpo di UNA regola è un **problema di ricerca** nello spazio combinatorio di tutti i letterali possibili, a rischio di esplosione combinatoria. Due strategie opposte:

- **Top-Down (Specializzazione)**: si parte dalla regola più generale possibile, “ $\oplus \leftarrow$ ” (copre tutti gli esempi, corpo vuoto), e si **aggiungono letterali** uno alla volta fino a coprire solo positivi. Esempio: si parte da $\text{ripe} \leftarrow$, poi si specializza in $\text{ripe} \leftarrow (\text{color} = \text{green})$, ecc.
- **Bottom-Up (Generalizzazione)**: si parte da un singolo campione positivo specifico (tutti i suoi valori di attributo in AND) e si **rimuovono letterali** progressivamente per aumentare la copertura.

A ogni passo, i letterali candidati vengono valutati con **accuratezza**, **information gain (ratio)** o **Gini index**. Poiché la scelta greedy del “miglior” letterale a ogni passo può portare a **ottimi locali** (regole frammentate, sub-ottimali), si usa la **Beam Search**: invece di tenere un solo candidato migliore, si mantengono i b migliori candidati a ogni round, esplorando più rami in parallelo.

Pruning Optimization

Per evitare overfitting delle singole regole si usano tecniche di potatura (pruning), sia preventive che successive.

Pre-pruning — LRS (Likelihood Ratio Statistics, Clark & Niblett 1989): si ferma l'aggiunta di letterali se la statistica scende sotto soglia:

$$LRS = 2 \cdot \left(\hat{m}_+ \log_2 \frac{\hat{m}_+ / (\hat{m}_+ + \hat{m}_-)}{m_+ / (m_+ + m_-)} + \hat{m}_- \log_2 \frac{\hat{m}_- / (\hat{m}_+ + \hat{m}_-)}{m_- / (m_+ + m_-)} \right)$$

dove $\hat{m}_+$, $\hat{m}_-$ sono i positivi/negativi coperti dalla regola e m_+ , m_- quelli nell'intero dataset: la LRS confronta la probabilità empirica di positivi coperti dalla regola con quella "a priori", misurando quanto la regola è informativa rispetto al caso.

Post-pruning — REP (Reduced Error Pruning, Brunk & Pazzani 1991): dopo aver generato la regola, si valutano **tutte** le possibili operazioni di potatura (rimuovere un letterale o l'intera regola) su un validation set, ripetendo finché le prestazioni migliorano. È molto accurato ma costoso, $O(m^4)$.

IREP (Furnkranz & Widmer, 1994) combina i due approcci: ogni volta che una regola r è generata, viene **immediatamente potata** con REP, invece di aspettare la fine, riducendo la complessità a $O(m \log^2 m)$.

IREP* (Cohen, 1995) migliora IREP usando una metrica di accuratezza più bilanciata: $\frac{\hat{m}_+ + (m_- - \hat{m}_-)}{m_+ + m_-}$ (positivi coperti più negativi correttamente esclusi, sul totale).

RIPPER (Cohen, 1995) estende IREP: (1) genera un rule set iniziale con IREP; (2) per ogni regola, genera una **regola sostitutiva** (rigenerata da zero) oppure la **specializza e poi generalizza** nuovamente; (3) confronta il rule set risultante con l'originale e mantiene il migliore; (4) ripete finché non c'è più miglioramento. RIPPER **rivisita l'intero rule set alla fine**, alleviando il problema della località/ottimo locale tipico dell'approccio greedy (che produce regole "frammentate", ottimizzate solo localmente).

First-Order Rule Learning — FOIL

Le regole proposizionali descrivono solo attributi di un singolo esempio, ma non esprimono **relazioni tra campioni diversi** (es. "un cocomero è più maturo di un altro"). Serve la logica del primo ordine: oltre ai dati attribute-value, si aggiunge una **relational background knowledge** (fatti relazionali tra coppie di esempi), es.:

$$\forall X \forall Y \text{ (riper}(X, Y) \leftarrow \text{curler_root}(X, Y) \wedge \text{hollower_umbilicus}(X, Y))$$

che si legge "X è più maturo di Y se la radice di X è più arricciata e l'ombelico di X è più incavato di quelli di Y" (i quantificatori si omettono quando il contesto è chiaro).

FOIL (Quinlan, 1990) applica Sequential Covering + strategia **Top-Down** al caso relazionale: i letterali candidati sono **tutte le combinazioni di predicati di background e variabili** (es. per $\text{riper}(X, Y) \leftarrow$: $\text{darker_color}(X, Y)$, $\text{darker_color}(Y, X)$, $\text{curler_root}(X, Y)$, ecc.), scegliendo quello che massimizza il **FOIL Gain**:

$$F_GAIN = \hat{m}_+ \left(\log_2 \frac{\hat{m}_+}{\hat{m}_+ + \hat{m}_-} - \log_2 \frac{m_+}{m_+ + m_-} \right)$$

Il FOIL Gain misura quanto l'aggiunta del letterale aumenta la probabilità che un esempio coperto sia positivo, pesata per il numero di positivi ancora coperti $\hat{m}_+$: premia letterali che aumentano la precisione senza perdere troppa copertura. Dopo la generazione, il rule set finale viene comunque **post-potato** (post-prune).

Cenni a Inductive Logic Programming (ILP)

L'**ILP** (Muggleton, 1991) generalizza ulteriormente, apprendendo completamente **clausole di Horn del primo ordine**; usa ancora Sequential Covering per il rule set, ma per la singola regola adotta una strategia **Bottom-Up locale** (non enumera tutti i candidati, evitando l'esplosione esponenziale della ricerca top-down al crescere della lunghezza della regola).

Lo strumento chiave è la **Least General Generalization (LGG)** (Plotkin, 1970): dati due letterali/termini, $LGG(t, t) = t$ se sono la stessa costante, mentre $LGG(s, t) = V$ (nuova variabile) se $s \neq t$ sono costanti diverse; si generalizzano due regole trovando i letterali con predicato comune e sostituendo le coppie di costanti diverse con nuove variabili, eliminando i letterali senza predicato in comune. L'idea è ottenere la regola **più generale possibile con il minimo cambiamento** rispetto alle regole di partenza. La **RLGG** (Relative LGG, Plotkin 1971) tiene conto anche della background knowledge, inizializzando la regola più specifica per un esempio e come $e \leftarrow K$, dove K è la congiunzione di tutti gli atomi noti.